LIFE OS

Ontology v0.1 Entry Point

Bridge from checkpoint to the next design cycle

Purpose	Define the constraints, candidate primitives, ambiguity set, stress tests and investigation order needed to build Ontology Minimum v0.1 without losing the behavior discovered so far.

Status	Open design frontier
Reading mode	Continue working
Canonical sources	Foundation 02; Models 03-08; Case Studies 09; Rationales 10; Design Log 11
Checkpoint boundary	This is the final document before Ontology Minimum v0.1

No ontology decision in this document should be read as finalized unless it is explicitly marked Established in the Decision Register.

0. Why ontology comes now

Earlier work deliberately started from lived needs, ceremonies and case studies rather than from a data model. That sequence produced enough repeated structure that an ontology can now be compressed from evidence instead of invented top-down.

BOUNDARY	This document does not define Ontology Minimum v0.1. It defines the constraints, ambiguity set and test sequence for creating it.

The ontology is successful only if it preserves the behavior already documented: low-friction capture, progressive structure, reuse, contextual state, derivation, resumability, flexible commitment and a revisable self-model.

1. Constraints inherited from the checkpoint

- Low friction: simple things must remain simple.
- Progressive structuring: content must be able to gain structure without forced migration rituals.
- User-controlled semantic promotion: the system may propose; the user decides when an Idea becomes a Project or a value becomes an Entity.
- Derivation: do not store facts that are reliably computable from canonical structure.
- Context separation: generic meaning and project-specific state cannot be conflated.
- Partial operationality: useful operations should emerge from available properties/relations.
- Multiple views: one underlying representation must support exploration, execution, ceremony and resume lenses.
- Uncertainty visibility: the ontology itself must distinguish settled and Working semantics.
- Maintenance budget: elegance that increases routine maintenance is a regression.

2. Candidate primitive set

Candidate	Why it is being considered	Status
Entity	Identity, independent reference, lifecycle, reuse, query and relations.	Working candidate
Relation	Semantic/contextual connection between things.	Working candidate
Property / Literal	Lightweight attached information before independent identity is useful.	Open boundary
Time	Cross-cutting placement, interval, ordering and temporal aggregation.	Working candidate

No candidate is protected by elegance. If a primitive can be eliminated without losing required behavior, remove it.

3. Candidate compositions - not primitives yet

Construct	Current interpretation
Concept	Knowledge-oriented Entity candidate.
Project	Entity with persistent intention, evolution, decomposition and operational memory.

Construct	Current interpretation
Idea	Light intentional content/entity that may remain loose or be promoted.
Task	Entity whose available properties enable selected operational behavior.
Session	Entity/construct sampling lived activity in time.
Temporal Period	Interval used for aggregation/planning.
Phase	State/focus/stage attached to development; explicitly not equal to Temporal Period.
Goal / Milestone / Scenario	Intentional structures with different scopes; exact identity/composition still Open.
Practice	Process organizing repeated Sessions.
Ceremony	Maintenance Session-like process with a structured workflow.

4. Ambiguity A - What qualifies as Entity?

Working definition: something has enough independent identity to be referred to, found, linked and potentially deepened on its own. That definition is useful but still too semantic for implementation.

Promotion heuristic

- Independent reuse from multiple contexts.
- Multiple incoming/outgoing relations.
- Substantial nested structure.
- Independent lifecycle or state.
- Independent operations or query needs.
- Need for low-friction resume on its own.
- Need to reference it directly rather than only through its parent.

Questions for Entity

- Can an Entity exist with only a name/body and no declared type?
- Is Entity identity globally stable across structural reorganization?
- Can a relation itself be promoted to Entity if it gains lifecycle/structure?
- How do we preserve the original simple content when promotion occurs?
- What is the minimum required metadata, if any?

5. Ambiguity B - Property vs Relation vs Literal

The current model wants lightweight properties, but a relational ontology could model many properties as edges to values. The design question is not philosophical purity; it is what representation best preserves low friction and progressive promotion.

Option	Benefit	Risk
Property as primitive	Very cheap simple values; natural capture.	Potential dual model with Relation and harder promotion semantics.

Option	Benefit	Risk
Relation + Literal only	Unified graph semantics.	May over-formalize trivial data and make literals feel like fake entities.
Property as syntax over Relation/Literal	Simple UX with unified underlying semantics.	Requires clear rules for metadata, promotion and querying.

TEST	Use examples like Goal="make a useful shared-resource app", status, a free-text observation, an image caption and a person's current interest intensity. The chosen model must keep trivial values trivial while allowing deepening.

6. Ambiguity C - Relation and contextual metadata

Relations carry much of the model's expressive power: composed-of, is, requires, influences, uses, resonates-with, interested-in and many others. Some relations also need contextual state.

Questions for Relation

- Are relation types Concepts/Entities themselves or schema-level symbols?
- Which metadata belongs directly to the edge: relevance, blocker state, confidence, temporal validity, source?
- When does rich edge metadata indicate that the relation deserves its own Entity identity?
- How do derived/transitive relations remain explainable?
- Can relation semantics be progressively structured like Entity content?

7. Ambiguity D - Schema -> Capability -> Operation

This is a central design direction: rather than tagging an Entity as Operational and forcing a complete schema, individual properties/relations should unlock useful behaviors.

```
has status -> can transition state has scheduled time -> can appear in planning/calendar views has duration -> can match available slots has temporal placement -> can aggregate over periods has metric -> can support quantitative trend analysis
```

Questions

- Is a Capability explicit metadata, an inferred rule, or both?
- Can multiple alternative property sets enable the same Capability?
- Where do Operations live: ontology, application layer, view layer?
- How do we expose capabilities without making the Entity schema noisy?
- How are capability rules versioned as the system evolves?

8. Ambiguity E - Time

Time is cross-cutting and risks producing too many overlapping constructs. The ontology must support at least temporal placement, duration/interval, planning windows, aggregation and phase histories.

Construct	Must preserve
Session	A sampled lived activity/experience; not every micro-event.
Temporal Period	An interval used to collect/aggregate intention and reality.
Phase	State/focus/stage; can cross periods; explicitly distinct from period.
Commitment	Intentional relation between an Entity and a relevant period, potentially with target/weight.
Recurrence	Still underexplored; necessary for routines, maintenance, ceremonies and practice expectations.

9. Derived information

The ontology must make derivation a first-class design consideration even if computation happens later in implementation.

- Project completion relative to current decomposition/vision.
- Knowledge mastery/completion relative to current known substructure.
- Attention distribution from Sessions/activities.
- Inactivity from last meaningful interaction plus importance/commitment context.
- Commitment mismatch from intended allocation vs observed behavior.
- Transitive knowledge relevance such as Solducci -> App Mobile -> regulatory obligations.

RULE	A derived value should be reproducible/explainable from canonical facts. If it cannot be explained, it should not silently become authoritative.

10. Stress-test sequence

Do not validate primitives in isolation. For each proposed representation, run the same sequence:

1. Solducci: Can generic mobile-app knowledge, project-specific blocker state, evolving vision, ideas/features and goal decomposition coexist without duplication?
2. Speedcubing: Can Practice, Session, Metrics, Observations, Knowledge composition and derived progression be expressed without recording each solve?
3. Crochet: Can a loose inspirational connection mature into an Idea/Project and later yield reusable Pattern Knowledge without copying content?
4. Agrivoltaico: Can long-lived low-commitment Exploration, scenarios, knowledge gaps and parking remain coherent for years?
5. LaGenteStrana / HistoryExplorer: Do collaboration and low-intensity projects require any primitive not already justified?

11. Secondary-domain test suite

After the strong case studies pass, explicitly test domains that were underrepresented during discovery:

Domain	Questions to force
Health	Recurring checks, appointments, sensitive observations, maintenance tasks and non-project goals.

Domain	Questions to force
Relationships	Ongoing connection without turning people into optimization targets; reminders vs lived spontaneity.
Bureaucracy / finance	Deadlines, documents, dependencies, recurring obligations and low-interest but high-importance work.
Travel	Temporary Project/Scenario/itinerary structures, time/place constraints, resources and collaborative planning.
Recurring maintenance	Cadence, expected frequency, skipped instances, debt and non-goal continuity.

12. Investigation order

1. Entity: identity, minimum metadata, promotion criteria.
2. Literal/raw content and Property vs Relation semantics.
3. Relation types, metadata, contextual state and possible relation promotion.
4. Schema -> Capability -> Operation formalization.
5. Time: temporal placement, interval, Session and Temporal Period.
6. Concept/Knowledge composition and generic/contextual application.
7. Task as capability composition.
8. Project as identity + intention + structure.
9. Goal, Milestone, Scenario and Trajectory.
10. Phase, Commitment and Ceremony outputs.
11. Self, Values, Interest and Resonance.
12. Only then: implementation/storage/UI choices.

13. Acceptance criteria for Ontology Minimum v0.1

Criterion	Pass condition
Coverage	All strong case studies can be represented without important information loss.
Parsimony	Few primitives; special cases require explicit justification.
Progressive complexity	Simple captured content does not require mature schemas.
Promotion continuity	A simple value can deepen without breaking references or requiring duplication.
Context safety	Generic knowledge stays reusable while contextual state remains local.
Derivability	Secondary state can be computed without redundant canonical storage.
Operational composability	Useful operations can emerge from partial properties/relations.
Resume quality	Paused initiatives retain enough structure for low-friction resumption.
Human authority	No ontology mechanism silently converts observed behavior into values, priorities or commitments.
Maintenance budget	The model does not demand routine data entry with no clear downstream benefit.

14. Primitive 1 - Entity: exact next discussion

NEXT QUESTION	What is the minimum meaning of Entity that gives us identity, referencing, lifecycle and promotion benefits - and which things we currently call entities can still be represented more cheaply as text, property or relation?

The next design turn should answer that question using concrete examples before defining fields or storage. The ontology should continue to be discovered from behavior, not imposed from implementation preferences.